

SnapFFT

Nakul Kadam

13 July 2026

Introduction

This project is an application of four frequency transforms onto the live webcam footage of the device. Extensive use of 2D Fast Fourier Transform (fft2) to manipulate the raw webcam frame and apply desired frequency filter. The webcam capturing and output display is handled by python; while frequency domain manipulation of the V channel (brightness) is handled with MATLAB (session run internally by python).

Inspiration

Snapchat

Inspired by how Snapchat filters are applied to the camera frames instantly. Learnt about how Snapchat uses "Facial Landmark detection" to distinguish between human and animal faces.

Python libraries :

0.1 OpenCV (cv2)

Live webcam monitoring, window and output display, Trackbars, keyboard input, color interconversion (BGR and HSV), fps, snapshots.

0.2 numpy

Creating the FFT Magnitude spectrum, magnitude extraction, log scaling, array stacking, slicing, converting MATLAB array to numpy array.

0.3 Matlab engine for python (matlab.engine)

Starting and shutting a MATLAB session with the arguments from OpenCV trackbars and webcam, converting numpy array to MATLAB-compatible array (uint8), adding project folder to MATLAB's path.

MATLAB toolboxes :

Base MATLAB

2D Fast Fourier Transform functions, case switching and performing desired frequency domain manipulation. Mathematical tools used in the filter creation.

Workflow

0.1 Startup

Python launches a MATLAB engine session behind the screen and tells it where .m file lives. OpenCV opens the webcam at 640×480 and creates the display window with three trackbar sliders (filter radius, DC protect radius, intensity).

0.2 python to MATLAB

OpenCV converts current BGR webcam frame to HSV and extracts only the V (brightness) channel; since our FFT filter works on the V channel only. It reads the current slider values from OpenCV, then sends the V channel and all parameters to MATLAB.

0.3 MATLAB to python

MATLAB file receives the V channel as a matrix. For homomorphic mode it applies a log transform first. It runs `fft2` to convert to frequency domain, then `fftshift()` to center the DC component.

A 2D mask is constructed for the selected filter mode. Each filter shapes the mask differently using Gaussian Bell curve as the common building block. The mask is multiplied element-wise in the frequency domain (Blocking, passing, or boosting specific frequencies). `ifftshift()` and `ifft2()` reconstruct the spatial image.

The result is blended with the original at the user-controlled intensity level, clipped to 0–255, and returned to Python as `uint8`.

Filter modes

Sharpen

Boosts the edges without removing anything. Boundaries become crisp and gives sketch-like effect.

Homomorphic

Uneven illumination is handled at frequency level rather than spatial. The gap between bright and shadow part of the image is reduced. Shadows get lifted while highlights are toned down.

LowPass

Blocks all the fine details, gives gradient-like effect around edges and keeps only the broad shapes.

HighPass

The DC object is kept but details (gradient) are blocked; while edges are passed.

None

Shows the live webcam without any filter.

Function	Use
<i>Python (OpenCV / NumPy / MATLAB Engine)</i>	
<code>cv2.cvtColor()</code>	Converts the webcam frame from BGR to HSV and filtered HSV back to BGR
<code>cv2.normalize()</code>	Stretches FFT magnitude values into the 0–255 range so the spectrum can be displayed as an image
<code>cv2.applyColorMap()</code>	Maps the grayscale FFT magnitude to the MAGMA color palette for the spectrum display panel
<code>cv2.putText()</code>	Overlays panel labels and FPS counter directly onto each of the four display panels
<code>cv2.waitKey()</code>	Waits for keyboard input each frame; also triggers the OpenCV window to refresh and render
<code>cv2.getTickCount()</code>	Records a high-resolution timestamp before and after each frame to compute live FPS
<code>np.fft.fft2()</code> , <code>np.fft.fftshift()</code>	Computes 2D Fast Fourier Transform of the V channel for the magnitude display and recentres the DC component
<code>np.abs()</code>	Extracts the magnitude of complex FFT output, representing how strongly each frequency is present
<code>np.log1p()</code>	Applies $\log(1+\text{input})$ to FFT magnitude to avoid $\log(0)$ case
<code>np.hstack()</code> / <code>np.vstack()</code>	Assembles the four 320×320 panels into the final 640×640 two-by-two display grid
<code>matlab.uint8()</code>	Converts the NumPy V channel array into MATLAB-compatible unsigned 8-bit format before transfer
<code>matlab.engine.start_matlab()</code>	Launches a full MATLAB session in the background at startup for the engine bridge
<i>MATLAB</i>	
<code>fft2()</code> , <code>fftshift()</code> , <code>ifftshift()</code> , <code>ifft2()</code>	Transforms V channel from spatial domain into frequency domain. Moves DC component to centre. Shift back DC component to original position after masking. Reconstruct spatial image of the masked image
<code>log1p()</code> / <code>expm1()</code>	Applies and reverses the log transform wrapping the FFT pipeline for the homomorphic filter
<code>exp()</code>	Used in the Gaussian bell curve formula inside <code>gaussian_rolloff</code> to build smooth frequency masks
<code>strcmpi()</code>	Case-insensitive string comparison used to safely switch during Homomorphic case
<code>ones()</code>	Initialises the notch mask as fully open (all ones) before selectively suppressing frequencies
<code>real()</code>	Removes residual imaginary components left by Floating Point Arithmetic after IFFT

Table 1: Key functions used in the SnapFFT project